```python
from flask import Flask, render_template, request
import pandas as pd
import matplotlib.pyplot as plt
import os

from sklearn.datasets import load_iris, load_digits, load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

app = Flask(__name__)
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)


# Load built-in datasets

def load_dataset(name):
    if name == "iris":
        data = load_iris()
        return data.data, data.target
    elif name == "digits":
        data = load_digits()
        return data.data, data.target
    elif name == "wine":
        data = load_wine()
        return data.data, data.target


# Load model

def load_model(name):
    if name == "logistic":
        return LogisticRegression(max_iter=500)
    elif name == "knn":
        return KNeighborsClassifier(n_neighbors=5)
    elif name == "svm":
        return SVC()

@app.route("/", methods=["GET", "POST"])
def home():

    results = None
    dataset_used = None

    if request.method == "POST":

        model_name = request.form["model"]
        dataset_type = request.form["dataset_type"]
        model = load_model(model_name)


        # Case 1: Built-in dataset

        if dataset_type == "builtin":
            dataset_name = request.form["dataset"]
            X, y = load_dataset(dataset_name)
            dataset_used = dataset_name.upper()

        # Case 2: Uploaded CSV

        else:
            file = request.files["csv_file"]
            df = pd.read_csv(file)

            # Assume last column is target
            X = df.iloc[:, :-1].values
            y = df.iloc[:, -1].values
            dataset_used = "Uploaded CSV"


        # Train-test split

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )


        # Feature Scaling

        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train)
        X_test = scaler.transform(X_test)


        # Train & Predict

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)


        # Metrics

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred, average="weighted")
        rec = recall_score(y_test, y_pred, average="weighted")
        f1 = f1_score(y_test, y_pred, average="weighted")


        # Plot Metrics

        metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
        values = [acc, prec, rec, f1]

        plt.figure()
        plt.bar(metrics, values)
        plt.ylim(0, 1)
        plt.title(f"Performance on {dataset_used}")
        plt.savefig("static/metrics.png")
        plt.close()

        results = {
            "accuracy": acc,
            "precision": prec,
            "recall": rec,
            "f1": f1,
            "dataset": dataset_used
        }
```

```
    return render_template("index.html", results=results)

if __name__ == "__main__":
    app.run(debug=True)
```